

Capitolo 2

Agenti Intelligenti

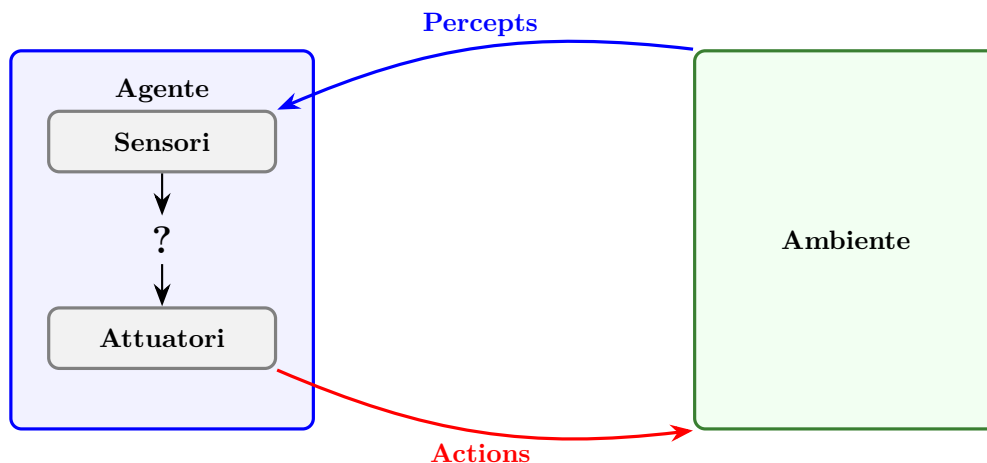
Lezione 2 – 03/03/2026 – Russell & Norvig, Cap. 2 (§2.1–2.4)

L'idea di **agente razionale** è il filo conduttore di tutto il libro di Russell & Norvig: invece di chiederci se una macchina “pensa”, ci chiediamo se *agisce nel modo migliore possibile* per raggiungere i propri obiettivi, data l'evidenza disponibile.

2.1 Agenti e interazione con l'ambiente

Agente

Un **agente** è qualsiasi entità che *percepisce* il proprio ambiente tramite **sensori** e *agisce* su di esso tramite **attuatori**.



Esempi di agenti:

- **Agente umano:** sensori = vista, udito, tatto, olfatto; attuatori = mani, piedi, voce.
- **Agente robotico:** sensori = telecamere, infrarossi, sonar, accelerometri; attuatori = motori, pistoni, altoparlanti.
- **Agente software (*softbot*):** riceve input da file, rete, tastiera, API; agisce scrivendo file, inviando pacchetti di rete, visualizzando informazioni.

Esempi pratici

Agricoltura intelligente: un sensore di acidità del terreno percepisce i dati per regolare fertilizzanti e irrigazione (attuatori).

Domotica: un sensore di illuminazione naturale attiva l'abbassamento/alzamento delle tapparelle o l'accensione del riscaldamento.

2.1.1 Percetti, sequenza percettiva e funzione agente

- **Percetto:** il contenuto percepito dai sensori in un dato istante.
- **Sequenza percettiva:** la storia completa di tutto ciò che l'agente ha mai percepito.

La scelta dell'azione può dipendere dalla conoscenza pregressa e dall'intera sequenza percettiva, ma *non* da ciò che l'agente non ha percepito.

Funzione agente e Programma agente

La **funzione agente** $f : \mathcal{P}^* \rightarrow \mathcal{A}$ è la descrizione matematica (astratta) che mappa ogni possibile sequenza percettiva in un'azione.

Il **programma agente** è la sua implementazione concreta, eseguita su un'**architettura** fisica (hardware con sensori e attuatori).

$$\text{Agente} = \text{Architettura} + \text{Programma}$$

2.1.2 Il mondo dell'aspirapolvere**Esempio canonico: il mondo dell'aspirapolvere (Russell & Norvig)**

Due caselle (A e B), ciascuna pulita o sporca. Percetti: posizione + stato della casella. Azioni: **Aspira**, **Sinistra**, **Destra**.

Funzione agente semplice (agente reflex):

1. Se la casella corrente è sporca $\rightarrow$ **Aspira**.
2. Se la casella è pulita e l'agente è in $A \rightarrow$ **Destra**.
3. Se la casella è pulita e l'agente è in $B \rightarrow$ **Sinistra**.

Misura di prestazione corretta: un punto per ogni casella pulita ad ogni passo temporale, con penalità per spostamenti ed energia consumata – *non* “quantità di sporco rimossa” (altrimenti l'agente potrebbe spargere sporco per ripulirlo!).

Algoritmo del serpente

In ambienti più grandi (es. griglia 10×10), un agente reflex di base può implementare l'**algoritmo del serpente**: percorre ogni riga da sinistra a destra, poi scende di una riga e torna da destra a sinistra, e così via. È il metodo usato nei sistemi automatici di raccolta ordini nei magazzini. Non è però ottimale in ambienti dinamici dove la sporcizia riappare.

2.2 Razionalità e misure di prestazione

Agente razionale

Per ogni possibile sequenza percettiva, un **agente razionale** deve selezionare l'azione che ci si aspetta *massimizzi* la sua **misura di prestazione**, sulla base dell'evidenza fornita dalla sequenza percettiva e dalla propria conoscenza pregressa.

È meglio progettare le misure di prestazione in base a *ciò che si vuole davvero ottenere nell'ambiente*, non in base a come si pensa che l'agente dovrebbe comportarsi.

2.2.1 I 4 pilastri della razionalità

La razionalità dipende da quattro fattori concomitanti:

1. **La misura di prestazione**: il criterio di successo (es. pulizia media nel tempo; valutazione dell'aspirapolvere in base alla pulizia raggiunta rispetto alla sporcizia totale; la qualità è data anche dalla quantità di spostamenti necessari per pulire l'ambiente).
2. **La conoscenza pregressa**: ciò che l'agente conosce già dell'ambiente prima di agire.
3. **Le azioni eseguibili**: il repertorio di operazioni permesse dagli attuatori.
4. **La sequenza percettiva**: la storia completa di tutto ciò che l'agente ha percepito.

2.2.2 Razionalità vs. onniscienza e autonomia

La razionalità **non** è perfezione né onniscienza.

- Un agente **onnisciente** conosce l'esito reale delle azioni – ma l'onniscienza è impossibile nella realtà.
- La **razionalità** massimizza la prestazione *attesa*; la **perfezione** massimizza la prestazione *effettiva*.

Un agente dovrebbe scegliere un'azione che fornisca le migliori prestazioni sulla base del percepito e della conoscenza che ha (es. sa quali sono le dimensioni del mondo in cui si trova?). La scelta razionale di una persona può sembrare irrazionale a un'altra, se questa non è a conoscenza del suo punto di vista: *perché hai fatto così o non colà?*

Non basta raccogliere le informazioni, bisogna percepirle. Qui subentrano i **limiti della razionalità**:

- **Filosofia e libero arbitrio**: La razionalità ha un limite che richiama interpretazioni filosofiche e teologiche sul libero arbitrio. L'uomo può scegliere, quindi è autonomo. Se non avesse possibilità di scelta, sarebbe un burattino nelle mani dell'onnisciente nella ricerca del bene.

Scelta vincolata vs. libera

Il treno si muove vincolato sulle rotaie (nessuna scelta), l'auto si può muovere su diversi percorsi scelti.

- **Conoscenza incompleta:** L'agente non ha sempre la completa conoscenza dell'ambiente.

Auto a guida autonoma

L'auto a guida autonoma e il bimbo che attraversa la strada all'improvviso. In questi casi, l'autonomia di giudizio completa la parziale conoscenza.

- **Onniscienza limitata:** Un agente onnisciente conosce l'esito delle sue azioni e sceglie quella per il miglior risultato.

Giochi a turni

Nel gioco del tris o degli scacchi, l'onniscienza è per forza parziale/limitata perché non si ha la certezza della mossa successiva dell'avversario.

Razionale ma sfortunato (R&N)

Attraversi la strada dopo aver guardato in entrambe le direzioni. Un carico cade da un aereo di passaggio e ti colpisce. Hai agito *razionalmente* (la tua scelta era ottima data l'evidenza disponibile), ma il risultato è stato pessimo. La razionalità si giudica *ex ante*, non *ex post*.

Un agente razionale deve anche:

- **Raccogliere informazioni** (*information gathering*): guardare prima di attraversare, esplorazione di ambienti ignoti.
- **Apprendere** dall'esperienza per compensare conoscenza pregressa parziale o errata.
- Essere **autonomo**: il comportamento deve essere determinato dalla propria esperienza, non solo dalla conoscenza pre-programmata dal progettista.

Autonomia

Un agente che si affida solo alla conoscenza pre-programmata *manca di autonomia*. Così come l'evoluzione fornisce agli animali riflessi innati per sopravvivere abbastanza a lungo da imparare, un agente artificiale può partire con una conoscenza iniziale ma deve acquisire autonomia attraverso l'apprendimento.

2.3 La natura degli ambienti: PEAS

PEAS – Task Environment

Per progettare un agente, il primo passo è specificare il **task environment** nel modo più completo possibile. Si usa l'acronimo **PEAS**:

- **Performance measure** – criterio di successo
- **Environment** – l'ambiente esterno
- **Actuators** – gli attuatori disponibili
- **Sensors** – i sensori disponibili

PEAS per un taxi autonomo

- **P:** viaggio sicuro, veloce, legale, confortevole; massimizzare i profitti.
- **E:** strade, altri veicoli, pedoni, segnali stradali, meteo, clienti.
- **A:** sterzo, acceleratore, freni, frecce, clacson, display di bordo.
- **S:** videocamere, lidar, sonar, GPS, tachimetro, accelerometri, microfono.

2.3.1 Dimensioni di classificazione degli ambienti

Dimensione	Opzioni	Descrizione ed esempi
Osservabilità	Tot. osservabile	I sensori dell'agente gli consentono di accedere allo stato completo dell'ambiente in ogni momento (es. sensore di temperatura, scacchi)
	Parz. osservabile	I sensori dell'agente sono rumorosi o imprecisi (es. guida di un taxi)
	Inosservabile	L'agente non ha sensori
Numero di agenti	Singolo	Agente unico (es. cruciverba)
	Multi-agente	Più agenti interagiscono e i loro comportamenti si influenzano a vicenda (es. auto a guida autonoma)
Determinismo	Deterministico	Stato successivo certo (es. l'aspirapolvere pulisce la cella sporca)
	Stocastico	Subentra la probabilità (es. sacchetto dell'aspirapolvere pieno)
Episodicità	Episodico	L'azione è divisa in episodi atomici e indipendenti (es. passi di una ricetta)
	Sequenziale	Le azioni sono legate (es. parole crociate)
Dinamicità	Dinamico	L'ambiente evolve nel tempo (es. cesta dei panni sporchi)
	Statico	L'ambiente non cambia mentre l'agente delibera (es. esame superato)
	Semi-dinamico	L'ambiente non cambia ma la prestazione nel tempo sì (es. scacchi)
Discrezione	Discreto	Stati ben definiti (es. tiro di un dado, numeri al lotto)
	Continuo	Variabili continue nello spazio e nel tempo (es. guida di un taxi)
Notorietà	Noto	Esiti conosciuti (es. lancio di un dado)
	Ignoto	Esiti sconosciuti (es. poker con bluff ma esito ignoto)

L'ambiente più difficile

L'ambiente più complesso è: *parzialmente osservabile, multi-agente, stocastico, sequenziale, dinamico, continuo, ignoto* – il mondo reale. La guida del taxi è difficile su tutte queste dimensioni tranne che la conoscenza dell'ambiente (le leggi della fisica e del traffico sono note).

La tabella seguente riassume la classificazione di alcuni ambienti tipici:

Task environment	Osservabile	Agenti	Determinismo	Episodico	Dinamistico	Discreto
Cruciverba	Totalmente	Singolo	Deterministico	Sequenziale	Statico	Discreto
Scacchi (con orologio)	Totalmente	Multi	Deterministico	Sequenziale	Semi-dinamico	Discreto
Poker	Parzialmente	Multi	Stocastico	Sequenziale	Statico	Discreto
Guida del taxi	Parzialmente	Multi	Stocastico	Sequenziale	Dinamico	Continuo
Diagnosi medica	Parzialmente	Singolo	Stocastico	Sequenziale	Dinamico	Continuo
Analisi immagini	Totalmente	Singolo	Deterministico	Episodico	Semi-dinamico	Continuo

2.4 Struttura e tipologie degli agenti

Il **programma agente** implementa la funzione agente (la mappatura percetti $\rightarrow$ azioni). Esistono quattro architetture di base, in ordine crescente di complessità.

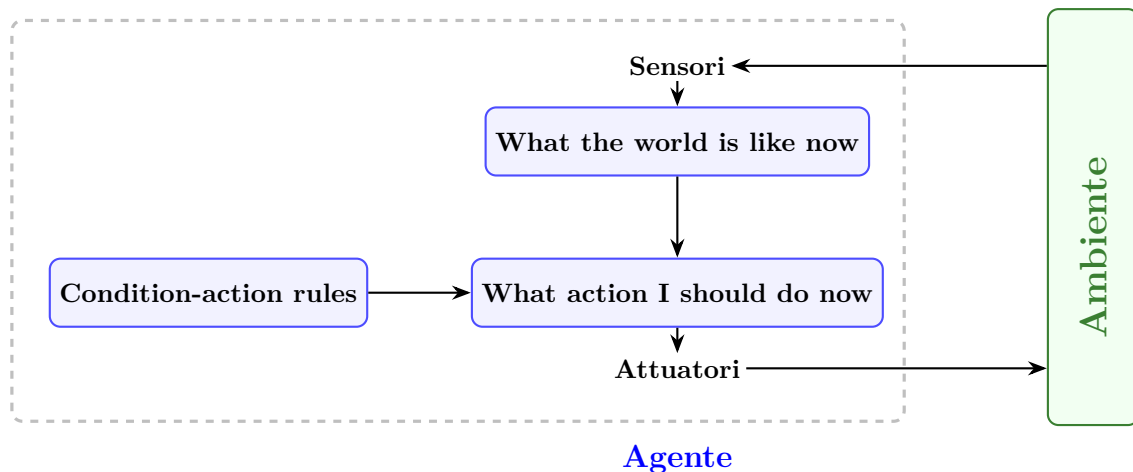
2.4.1 1. Agenti Reflex Semplici

Sono la tipologia più basilare di agente intelligente: selezionano le azioni basandosi esclusivamente sulla **percezione corrente**, ignorando completamente lo storico delle percezioni passate.

Si basano su **regole condizione-azione** (*condition-action rules*) espresse nella forma **if [condizione] then [azione]**. Il processo avviene in 3 fasi:

1. **Percezione:** i sensori catturano i dati dall'ambiente.
2. **Elaborazione:** l'agente converte la percezione in una descrizione della situazione corrente.
3. **Corrispondenza:** quando la percezione soddisfa la "condizione" di una regola, viene attivata l'"azione" associata.

L'agente reflex è razionale *solo* se l'ambiente è completamente osservabile. Il limite principale è la **parziale osservabilità**, che può provocare un **loop infinito** in cui l'agente oscilla tra gli ambienti e spreca energia. Un esempio è il robot aspirapolvere base che agisce solo alla presenza di sporco nella cella in cui si trova in quel preciso istante, senza ricordare le stanze già pulite.



Il flusso decisionale dell'agente reflex:

1. I sensori ricevono una percezione dell'ambiente in un dato istante.
2. L'agente interpreta l'input sensoriale per definire la situazione corrente.
3. Applicazione delle regole condizione-azione.
4. Viene selezionata l'azione corrispondente alla percezione attuale.
5. L'azione viene inviata agli attuatori per essere eseguita.

Pseudocodice agente reflex aspirapolvere

```

function REFLEX-VACUUM-AGENT([location, status]) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
  
```

- **Vantaggi:** estrema semplicità, basso costo computazionale, efficienza computazionale, semplicità di implementazione, basso consumo di risorse.
- **Limiti:** funzionano *solo* in ambienti completamente osservabili. In ambienti parzialmente osservabili, l'agente può cadere in **loop infiniti** (es. aspirapolvere senza sensore di posizione che oscilla tra *A* e *B*). Inoltre, manca di **adattabilità**: non può imparare dai propri errori e fallisce in ambienti dinamici.
- **Soluzione parziale:** lo spostamento casuale (*random*) dell'agente in un altro ambiente permette di uscire dal loop senza dover memorizzare lo stato passato, ma non è ottimale.
- **Nota:** l'agente reflex non mantiene alcuna memoria delle percezioni passate, quindi ogni decisione è basata esclusivamente sull'input sensoriale corrente.
- **Applicazioni tipiche:** termostati, rilevatori di fumo, sistemi di controllo semplici dove l'ambiente è completamente osservabile e le regole sono ben definite.

2.4.2 2. Agenti Reflex Basati su Modello

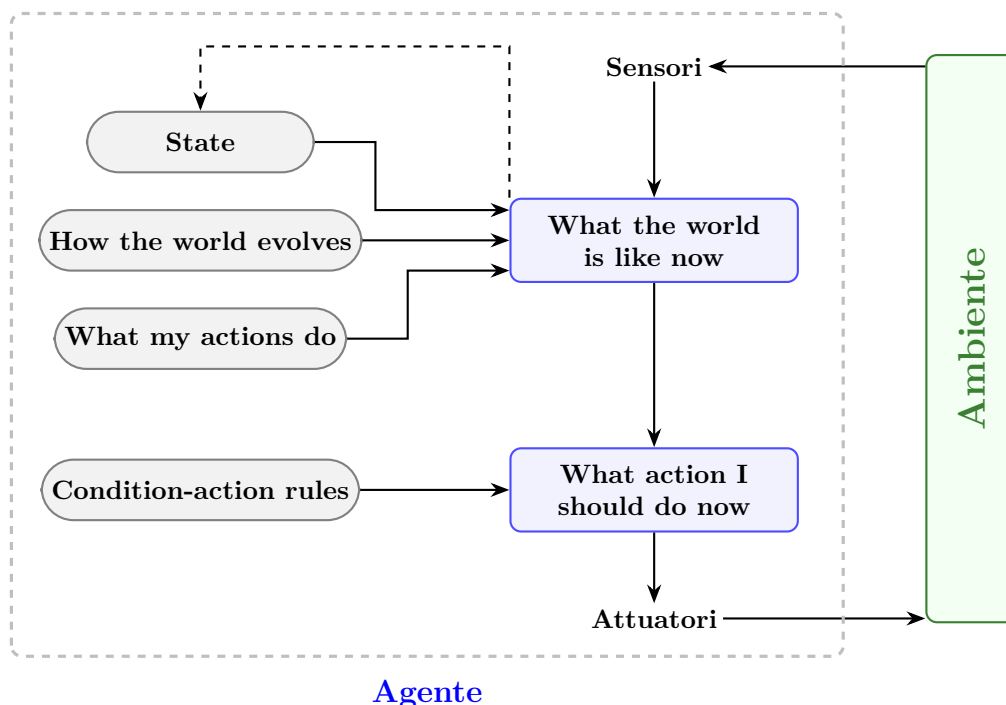
L'agente **model-based** è stato progettato per gestire la **parziale osservabilità** dell'ambiente mantenendo uno **stato interno** che tiene traccia della parte di mondo che l'agente non può vedere in quel momento. Questo stato dipende dalla storia delle percezioni e riflette gli aspetti inosservabili dello stato attuale.

L'agente conosce:

- Lo **stato** che rappresenta la memoria dell'agente.
- **Modello di transizione**: la conoscenza di come il mondo evolve nel tempo in modo indipendente dall'agente, e di come le azioni dell'agente stesso modifichino l'ambiente (effetti delle azioni + evoluzione autonoma). Descrive come cambia l'ambiente e gli effetti delle azioni dell'agente sul mondo.
- **Modello sensoriale**: come lo stato del mondo si riflette nelle percezioni dei sensori. Descrive come lo stato del mondo si rifletta nelle percezioni dell'agente.

Robot aspirapolvere model-based

Il robot ricorda che dietro di sé c'è un muro, anche se i sensori frontali non lo vedono in questo momento. Lo stato interno conserva la mappa parziale dell'ambiente esplorato.



Il flusso decisionale dell'agente model-based:

1. **Percezione**: i sensori ricevono dati dall'ambiente.
2. **Aggiornamento dello stato**: l'agente determina com'è il mondo ora per colmare le parti non visibili, combinando lo stato precedente, l'ultima azione compiuta, la percezione attuale e i modelli di transizione e sensoriale.
3. **Applicazione delle regole** condizione-azione.
4. **Decisione**: scelta dell'azione che l'agente deve eseguire cercando una regola corrispondente allo stato attuale.
5. **Esecuzione**: l'azione viene inviata agli attuatori per influenzare l'ambiente.

Pseudocodice agente model-based

```

function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               model, a description of how the next state depends
                   on current state and action
               rules, a set of condition-action rules
               action, the most recent action, initially none

  state UPDATE-STATE(state, action, percept, model)
  rule  RULE-MATCH(state, rules)
  action rule.ACTION
  return action

```

2.4.3 3. Agenti Basati sull'Obiettivo

Oltre allo stato corrente, dispongono di **informazioni sull'obiettivo** (*goal*): descrivono situazioni desiderabili (es. trovarsi alla destinazione del passeggero). Il programma combina il modello del mondo con il goal per scegliere azioni che lo raggiungano. Il vantaggio è la **flessibilità**: contrariamente agli agenti riflessi che hanno regole fisse, se l'obiettivo cambia, il comportamento dell'agente si adatta automaticamente per raggiungere il nuovo obiettivo. Un esempio è il robot aspirapolvere il cui obiettivo è pulire tutta la scacchiera/casa e tornare alla base.

- Quando la selezione non è banale, si usano algoritmi di **ricerca e pianificazione**.
- **Vantaggio**: la conoscenza è rappresentata *esplicitamente* e può essere modificata. Per cambiare destinazione basta specificare un nuovo goal – un agente reflex dovrebbe riscrivere tutte le sue regole.

2.4.4 4. Agenti Basati sull'Utilità

I goal forniscono solo una distinzione binaria (“felice” o “infelice”). Spesso molte sequenze di azioni raggiungono l'obiettivo, ma alcune sono più rapide, sicure o economiche. Serve quindi uno strumento più preciso: la **funzione di utilità**.

Quando ci sono obiettivi in conflitto o incertezza sul successo di un'azione, l'agente non può limitarsi a chiedersi “raggiungo o non raggiungo il goal?”. Deve invece valutare quanto sia *buono* un determinato stato. La funzione di utilità assegna a ogni stato un valore numerico che rappresenta il “grado di soddisfazione” dell'agente in quello stato.

Un agente razionale basato sull'utilità sceglie l'azione che massimizza l'**utilità attesa**, cioè la media dei valori degli stati possibili, pesata per la probabilità che ciascuno si verifichi. In questo modo può gestire:

- **Obiettivi in conflitto**: ad esempio, raggiungere la destinazione nel minor tempo possibile ma anche consumare poca benzina.
- **Incertezza**: se un'azione può portare a esiti diversi con certe probabilità, l'agente sceglie quella con il miglior risultato atteso.

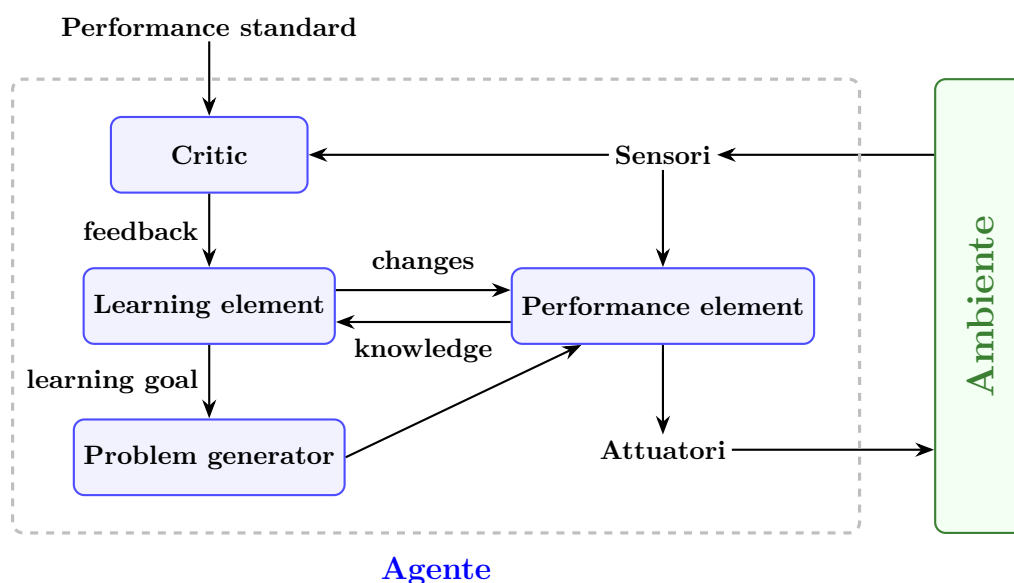
Auto a guida autonoma

L'obiettivo è arrivare a destinazione, ma la funzione di utilità combina: sicurezza (peso elevato), tempo di percorrenza, consumo di carburante, rispetto del codice della strada, comfort dei passeggeri. L'agente sceglie velocità e traiettoria che massimizzano l'utilità attesa.

2.4.5 5. Agenti che Apprendono

Qualsiasi tipo di agente (reflex, model-based, goal-based, utility-based) può essere dotato di **capacità di apprendimento**. L'apprendimento permette all'agente di operare in ambienti inizialmente ignoti e di migliorare le proprie prestazioni nel tempo.

Un **learning agent** è strutturato in quattro componenti:



- **Performance element:** l'agente vero e proprio – percepisce e agisce in base a ciò che conosce.
- **Critic:** valuta le prestazioni dell'agente rispetto a uno standard esterno fisso; passa il feedback al learning element.
- **Learning element:** responsabile dei miglioramenti – modifica la conoscenza del performance element.
- **Problem generator:** suggerisce azioni esplorative che potrebbero portare a esperienze informative (anche subottimali nel breve termine, migliori nel lungo).

Taxi che impara

Il performance element guida il taxi. Dopo una svolta brusca, il critic osserva la reazione degli altri guidatori (linguaggio colorito, frenate brusche) e passa il segnale negativo al learning element, che aggiunge la regola "svolta brusca = cattiva azione". Il problem generator potrebbe suggerire di testare i freni su asfalto bagnato per imparare la distanza di frenata.

2.4.6 Rappresentazioni dello stato

I componenti dell'agente possono rappresentare l'ambiente con livelli crescenti di espressività:

1. **Atomica**: ogni stato è indivisibile, senza struttura interna (es. nome di una città in un problema di percorso).
2. **Fattorizzata**: ogni stato è un vettore di variabili/attributi con valori (es. reti bayesiane, logica proposizionale).
3. **Strutturata**: oggetti con attributi e relazioni esplicite tra loro (es. logica del primo ordine, database relazionali).

Rappresentazioni **localiste** (un concetto = una locazione di memoria) vs. **distribuite** (un concetto = pattern su molte unità, tipiche delle reti neurali artificiali – più robuste al rumore).